

Architecture Design Document: E-Commerce Microservices System

1. System Overview

This project implements a distributed E-Commerce system based on a Microservices Architecture, developed using the .NET 10 framework. The system is designed to enforce a strict Separation of Concerns (SoC), high availability, fault tolerance, and independent scalability for each decoupled component.

The architecture consists of 5 core services:

ApiGateway: The single, secure reverse proxy entry point for all external client traffic, built using Microsoft's YARP (Yet Another Reverse Proxy) engine. •

ProductCatalogService: Manages the inventory catalog. This service is scaled out into two running instances (Replicas) to facilitate round-robin load balancing. •

OrderService: Handles customer checkout and orchestrates the order placement pipeline. •

InventoryService: Responsible for managing product stock levels and physical inventory reservations. •

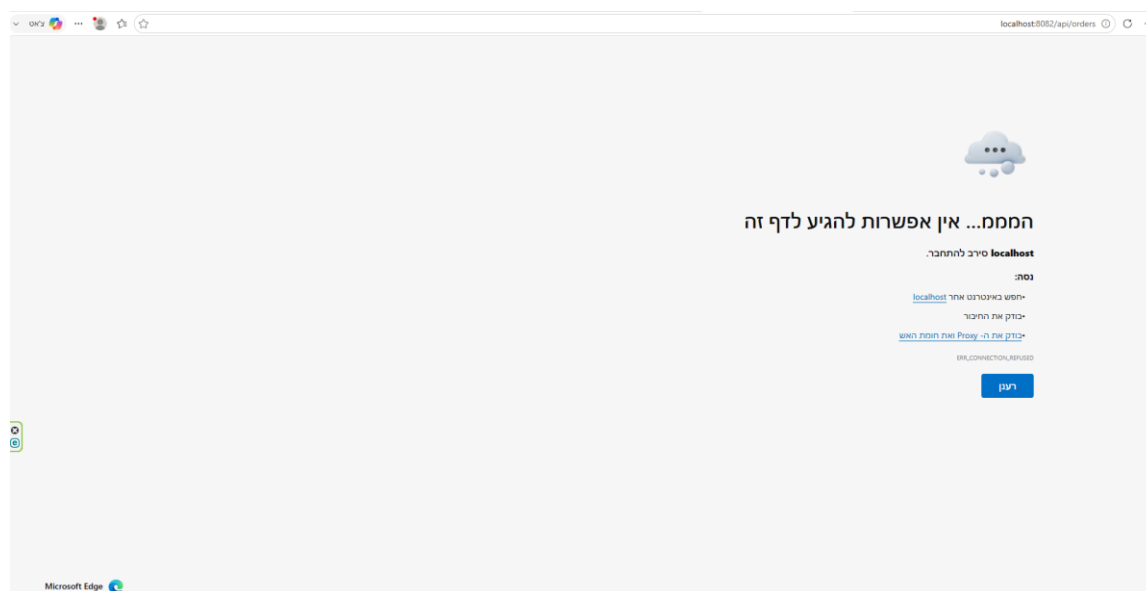
NotificationService: An isolated consumer service that asynchronously listens for and processes system-wide transactional alerts. •

2. Network Isolation, Security & Health Checks

The system is encapsulated within a custom Docker Compose network environment that enforces strict network isolation. All internal core microservices are entirely blocked from the public internet (no external ports are exposed in the compose configuration). All inbound client traffic is strictly routed through the ApiGateway.

Additionally, each service implements an automated health probing endpoint (/health), enabling continuous monitoring of the infrastructure's operational status.

Network Isolation and Perimeter Security Verification: The following image demonstrates a direct connection attempt to the inner OrderService (via the legacy external port 8082), which is successfully blocked by the firewall perimeter, resulting in a connection refusal:

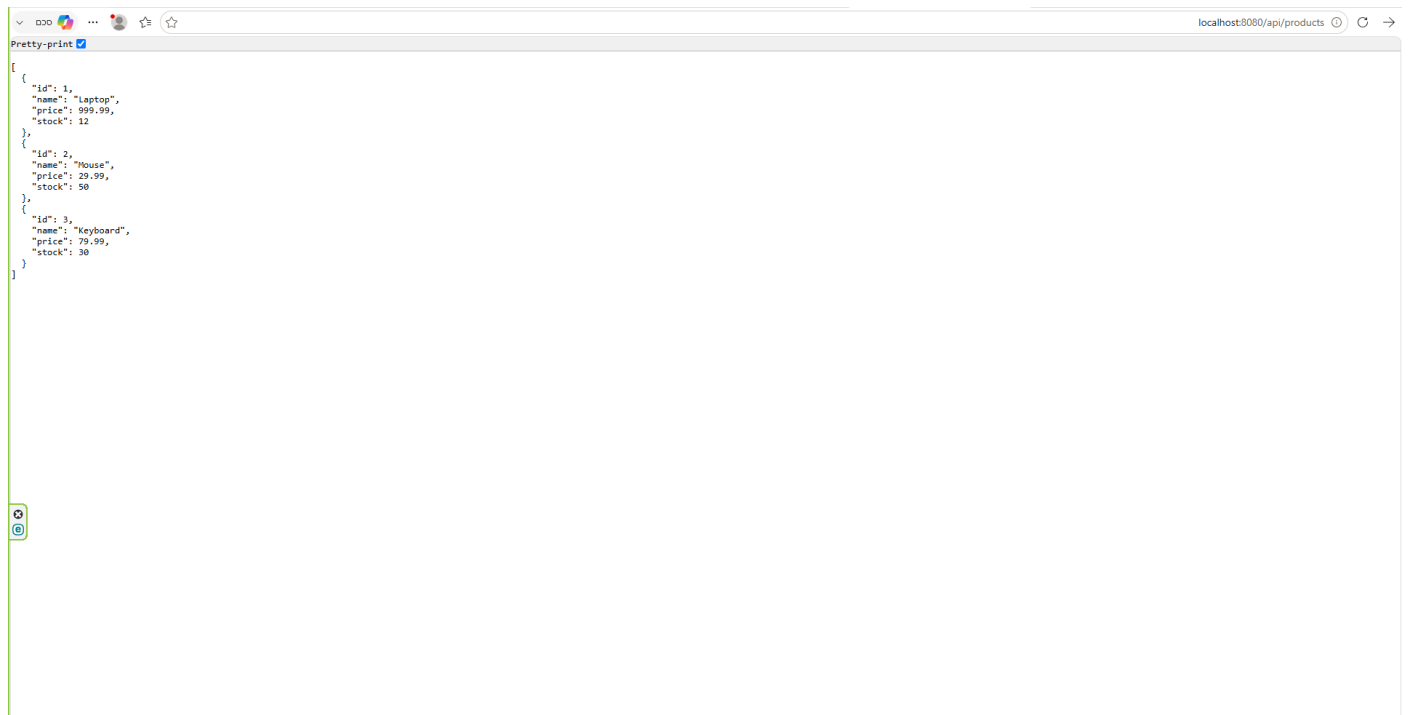


3. Request Routing, Load Balancing & Distributed Caching

The ApiGateway dynamically manages request forwarding and serves as an active Load Balancer for the ProductCatalogService across its redundant container replicas.

To optimize read throughput and minimize data store overhead, the catalog service implements the **Cache-Aside** design pattern utilizing a distributed **Redis Cache** cluster. Inbound product requests query the Redis cluster first; on a cache miss, the data is fetched from the primary store, cached with a 5-minute Time-To-Live (TTL) expiration, and returned to the client.

Gateway Routing and Active Catalog Caching Verification: The following image captures a successful HTTP GET request to the product catalog routed via the official API Gateway perimeter on port 8080:



4. Asynchronous Communication & Distributed Saga Orchestration

The transactional order placement workflow executes in a hybrid two-phase sequence to ensure eventual consistency and reliability across boundaries:

Synchronous Inter-Service Communication (HTTP): The OrderService triggers a synchronous REST .1 call to the InventoryService to reserve stock. If inventory is insufficient, the transaction is immediately aborted and the order is rejected.

Asynchronous Event-Driven Messaging: Upon successful stock reservation, the OrderService .2 publishes an internal event to an AMQP exchange routed to a queue named order-notifications on the **RabbitMQ Message Broker**. The NotificationService consumes this message asynchronously, completing the flow without blocking the primary user checkout thread.

End-to-End Saga Workflow and Message Broker Verification: The following view showcases a successful transaction response (201 Created with a Confirmed status) alongside the live terminal logs of the Notification container actively consuming and logging the broker event in real-time:

The screenshot displays a Visual Studio interface. The top pane shows a REST client request to `POST http://localhost:8080/api/orders` with a JSON body: `{ "productId": 1, "quantity": 2 }`. The response pane shows a `201 Created` status with headers including `Content-Type: application/json; charset=utf-8` and a JSON body: `{ "id": 2, "productId": 1, "quantity": 2, "status": "Confirmed" }`. The bottom pane shows a terminal window with Docker logs for `ecommerce`. The logs indicate that the `apigateway-1` service received the request and forwarded it to `orderservice-1`, which then processed the request and returned a `200` status.

5. Architectural Decision Defense

Database Engine Selection Strategy: The OrderService handles financial transactions, requiring strict ACID compliance and relational constraints provided by a SQL engine to guarantee data integrity. Conversely, the ProductCatalogService handles highly dynamic, polymorphic schema attributes with massive read-to-write ratios. This requirement aligns perfectly with a flexible NoSQL Document Model combined with distributed Redis caching to maximize read performance.

API Gateway Proxy Selection (YARP vs. Legacy Options): YARP (Yet Another Reverse Proxy) was selected over older alternatives like Ocelot due to its native .NET implementation, superior throughput architecture, and seamless integration with modern .NET configurations. YARP provides robust, built-in structural support for advanced routing, active/passive destination health parsing, and distributed load-balancing algorithms out of the box.